

Development of a Web-based Test Procedure Authoring and Execution Environment for Space Systems

Hongman Kim¹ and Christopher A. Swan²

Jet Propulsion Laboratory, California Institute of Technology, Pasadena, CA, 91109, United States

Well defined test procedures are essential to the successful execution of integration and tests campaigns of flight projects. Various forms of test procedures have been used that resulted in irregular and inefficient processes. Ingenium is a web-based procedure authoring and execution environment developed at JPL to provide a streamlined process around test procedures. It has been successfully used for major flight projects such as Psyche and Europa Clipper missions. This paper gives an overview of Ingenium and discusses how it delivers a solution that covers the entire lifecycle of test procedures including authoring, execution, and post-execution reviews. The capabilities and implementation approach of Ingenium are discussed along with technical challenges encountered. Use of state-of-the-art S/W technologies allowed creating a highly interactive environment that supports complex engineering processes of flight systems integration and tests.

I. Nomenclature

<i>AMMOS</i>	=	Advanced Multi-mission Operations System
<i>AMPCS</i>	=	AMMOS Mission data Processing and Control System
<i>API</i>	=	Application Programming Interface
<i>ATLO</i>	=	Assembly, Test, and Launch Operations
<i>CM</i>	=	Configuration Management
<i>EHA</i>	=	Engineering Housekeeping and Accountability packet
<i>EVR</i>	=	Event Verification Record
<i>FSW</i>	=	Flight Software
<i>GDS</i>	=	Ground Data System
<i>I&T</i>	=	Integration and Tests
<i>JPL</i>	=	Jet Propulsion Laboratory
<i>JWT</i>	=	JSON Web Token
<i>LDAP</i>	=	Lightweight Directory Access Protocol
<i>MTAK</i>	=	MPCS Test Automation Kit
<i>PBAT</i>	=	Procedures for Build, Assembly and Test
<i>R4R</i>	=	Run for Record
<i>REST</i>	=	Representational State Transfer
<i>SCMF</i>	=	Spacecraft Message File
<i>SSE</i>	=	Simulation Support Equipment
<i>UI</i>	=	User Interface
<i>V&V</i>	=	Verification and Validation
<i>VI</i>	=	Verification Item
<i>WSTS</i>	=	Workstation Simulation Test Sets

¹ Senior Systems Engineer, Systems Modeling, Analysis & Architectures, Senior Member of AIAA
² Senior Systems Engineer, Flight System Systems Engineering

II. Introduction

Written procedures are essential tools for the execution of complex engineering processes. NASA's deep space missions often require creation of one of a kind spacecraft and instruments. Thus, well-defined integration and test procedures are critical to the successful execution of integration and tests campaigns of such flight projects by rigorously testing and verifying the complex structure and behavior of such flight systems. For instance, thousands of test procedures are created to support the integration and tests of a flagship NASA mission. Creating test procedures requires lots of ground work by procedure authors, and they must understand the architecture and behavior of not only the system under test but also test support equipment such as simulation support equipment (SSE). Procedure authors also need to understand the scope of tests, and figure out the most effective way to carry out them, which will be translated to steps in test procedures.

Various forms of test procedures are used in aerospace industry. Figure 1 is a sampling of test procedures used by recent JPL flight projects. Paper procedures that are written in a document form have been widely used to date, where procedures are printed to paper, and test conductors write down test results on the paper copy. Paper procedures are very flexible since they do not enforce any particular ways of capturing and interpreting information, except for best practice or rules set by individual teams. Since procedure steps are executed manually, test conductors have a full control of the execution flow. At the same time, this manual nature of paper procedures is a source of inefficiencies and errors because test engineers may have to copy and paste flight commands and manually check telemetry. Paper procedures are more common in high level tests in later phases of system integration and tests such as ATLO (assembly, test, and launch operations).

On the other hand, scripted procedures are more common in earlier phases of integration and tests. Written in programming languages such as Python, scripted procedures can be executed in automatic mode, relieving test engineers of manual commanding and telemetry checks. Sophisticated test logics can be implemented as needed. However, creating scripted procedure requires a certain level of programming skills and they are more difficult to understand and review. Scripted procedures are typically executed in an automated fashion, and it is difficult to pause, resume, and jump around when needed unless certain manual/automatic switch and control logics are implemented. Many organizations use document-oriented configuration management (CM) systems, to which scripted procedures do not fit very well and they are typically managed in software-oriented CM systems.

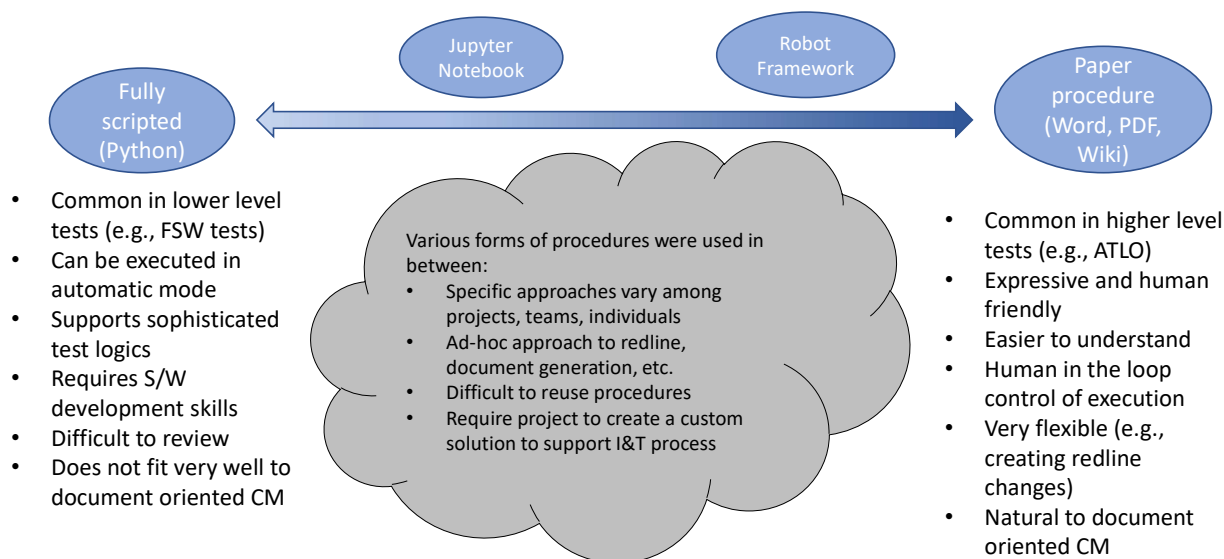


Figure 1: Various forms of test procedures.

To overcome some of the limitations of paper or scripted procedures, other forms of procedures have been also used at JPL. Early on, a TCL/TK based tool was developed to support automatic and manual execution of scripted procedures [1]. The capability was later extended to execute scripts embedded in procedures in MS Word format [2]. As Python script language gained more adoption, a Python library called MTAK [3] was developed that can interface

with ground data system and ground support equipment. The library has been widely used for flight software tests and avionics tests. In more recent years, Robot Framework [4], [5] was used to create a set of semi-automated tests for the ATLO operation of Mars Perseverance Rover. Robot framework supports the use of keyword-based steps that are translated to function calls in script languages such as Python. For some of the system level V&V tests, Jupyter notebooks [6], [7] have been also used, where Python codes can co-exist with textual instructions. In Jupyter notebooks, Python codes are organized by cells, which can be executed individually or in automatic mode, and the execution results can be captured in the same notebook.

While these forms of procedures were used successfully, specific approaches varied among projects, teams, and individual procedure authors. For example, various ad hoc approaches were used to capture redline changes (correction to a released procedure that will be reflected in the next release) or blueline changes (tactical and temporary modification of a released procedure). For certain forms of procedures, additional tooling was created to transform procedures to a document form that can be more easily reviewed by engineering staff and managed in traditional CM systems. In the absence of standard approaches, each team had to create a custom solution to support their I&T process, which made it difficult to reuse procedures across projects or even across teams in the same project.

Over the last six years, JPL developed a new procedure authoring and execution environment called Ingenium. It is a JPL internal suite of tools that offer baseline capabilities for the system level tests of flight systems. The goal is to provide a solution that can support the entire lifecycle of test procedures that includes creation, execution, and review of test procedures. Ingenium accomplished the goal by providing an electronic procedure capability that combines the best of the both worlds of paper procedures and scripted procedures. The electronic procedure capabilities of Ingenium enable creating test procedures in a standard format that is precisely defined. This makes it easy to understand procedures and interpret execution results. Use of Ingenium helped lower the learning curve of new test engineers who can create and execute procedures more easily through the web interface of Ingenium. Using procedures of the same electronic format also makes it easy for projects to reuse and share them.

The idea of e-procedure has been progressing in NASA for several years, particularly in the operation of human space flight [8], [9]. Paper procedures that were in use for the operation of International Space Station were converted to XML format with a precisely-defined schema. The e-procedures then can be transformed to various formats, and displayed in various computer devices including hand-held devices. The capability was extended to integrate with the command and telemetry system of flight vehicle, which allowed automatic commanding and telemetry verification from e-procedures. In more recent years, numerous start-up companies were created in aerospace industry, and some of them started using e-procedure capabilities for their operations and tests [10]. These examples of e-procedures are largely focused on the operation of space and launch systems. This paper discusses application of e-procedure to the system level integration and tests, which pose unique challenges to the authoring and execution of procedures. For example, test procedures may need to account for varying fidelity of system under test. In the following sections, the capabilities and implementation approach of Ingenium are discussed along with technical challenges encountered. It will be also discussed how it has been used to support two major NASA deep space missions and to improve JPL's integration and test capabilities.

III. S/W Architecture

Ingenium is a web application that consists of a number of microservices (Figure 2). At the highest level, there are two layers of applications: web application and Venue Service. A single instance of web application serves multiple instances of test venues. The user interface (UI) is a collection of single page applications that use VueJS framework [11]. All backend services implement RESTful APIs that are specified in OpenAPI [12] specifications. All of the web application services are containerized using Docker [13]. Depending on workload, the services can be deployed to a single host or to a cluster of hosts by using Docker Swarm [14] container orchestration. Here is a summary of Ingenium Web application services.

- Auth Service: provides user authentication and authorization that uses LDAP and JWT
- Dictionary Service: serves the definitions of FSW/SSE commands, telemetry, and verification items such as flight system requirements
- Archive Service: saves procedures and completed executions of procedures (as-runs)
- Core Service: coordinates procedure execution and data management
- Execution Service: executes procedure steps by sending commands to Ingenium Venue Service and checking telemetry

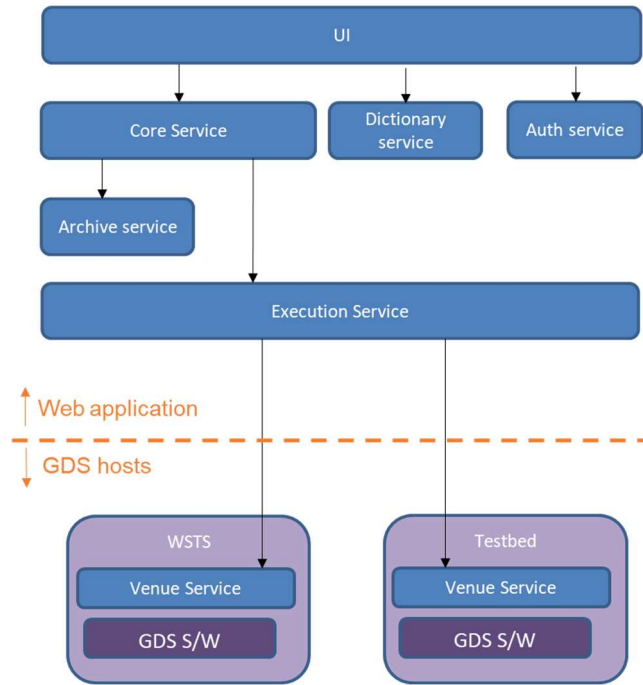


Figure 2: High level S/W architecture of Ingenium.

Ingenium interacts with system under test through a ground data system (GDS) called AMPCS [15]. Since the same GDS S/W is used for test venues of varying fidelity ranging from a simulation environment (e.g., WSTS), hardware testbeds, to ATLO venues, Ingenium communicates with various test venues in the same way. Ingenium can also send SSE commands via AMPCS. To interface with GDS S/W, Ingenium Venue Service is running on each of GDS hosts. Venue Service is secured in such a way it can be used only by a designated Ingenium instance with proper permissions.

IV. Capabilities and Implementation Approaches

In this section, capabilities of Ingenium are discussed in three function areas of procedure authoring, execution, and review. Capabilities that support testbed and ATLO operations are also discussed. Implementation approaches to the capabilities are discussed along with any technical challenges and architectural trades.

A. Procedure Authoring

Ingenium Steps

One of the goals of Ingenium was to provide an e-procedure capability that looks similar to paper procedure. An extensive review of existing paper procedures was conducted to define the scope of development and identify activities common in test procedures. Three organizational elements were defined: section, paragraph, and procedure element. Sections are used to group related activities and can be used to create a large procedure with a deeply nested structure. Paragraph is a text only element that is used to inform test engineers. Procedure element is used to refer to another procedure that needs to be executed as a child procedure. Functional activities such as sending command and checking telemetry are modeled as “step” elements. A single step can be used to send multiple commands or to check multiple telemetry channels. Currently, close to 20 steps are available in step palette in Ingenium. Below is a summary of elements in Ingenium.

Table 1: Procedure Constructs in Ingenium.

Category	Name	Description
Organization	Section	Container of steps or other sections
	Paragraph	Text only element that is used for information only

	Procedure	Reference to a version of another procedure that will be executed as part of the current procedure
Command	Command	Send one or more FSW command. The step may wait until the command is successfully processed by FSW.
	Command File	Send one or more file to FSW. If the file is a command file for immediate execution, it can be automatically executed by FSW.
	Command SCMF	Send one or more command file in SCMF format.
	Command SSE	Send one or more SSE command
Telemetry	Query EVR	Check the count of EVR telemetry (event message) for a given time range
	Wait EVR	Wait for EVR telemetry (event message) for its existence or non-existence.
	Verify EHA	Compare values of channelized telemetry against predicted conditions
	Wait EHA	Wait until values of channelized telemetry meets predicted conditions
	Wait/Verify 1553 Bus	Verify or wait for values in 1553 Bus traffic and compare against predicted conditions
	List Data Product	Check the list of data products that were downloaded against a predicted list
	Wait Data Product	Wait until data products are downloaded
	Time Reference	Used to create a named time reference that can be used to set query start or end time of later steps
	Wait	Wait for a predefined duration or until a predefined time
Manual	Venue Configuration	Used to record the current configuration of test such as GDS version, FSW/SSE dictionary versions, SSE version
	Manual GDS	Used to register GDS session ids that will be used for the test
	Manual Input	Used to record values that need to be obtained by test engineers manually
	Manual Verification	Used to describe activities that must be performed by test engineers and record its pass/fail status
	Environment	Used to record temperature and humidity of test venue
	Electronic Integration Procedure	Used to record voltage and/or resistance of circuit terminals during electronic integration

Description Field

Textual descriptions can be added to any element in Ingenium. A graphical text editor is used to add rich texts that may have lists, tables, and images. Descriptions are used to provide background information of a step, provide a list of actions for a manual step, describe fallback actions in the event of anomaly, or give warnings. Since the description is saved in HTML format, links to external resources can be also used. The description field allows the use of a free form text in e-procedures, which gives a feel like a paper procedure.

Dictionary Integration

Many functional tests of flight systems involve sending FSW/SSE commands and check telemetry. Dictionary Service was developed to provide dictionary lookup, which aids definition of command steps and telemetry steps. FSW/SSE dictionary files in XML format are ingested into Dictionary Service. Multiple versions of dictionaries can be supported at the same time, and users can select a version from the list when creating a procedure. The dictionary contents are stored in an Elastic Search database, which allows fast lookup via index search.

V&V Integration

Verification items (VI) are system or subsystem requirements that must be verified by test procedures. They are commonly maintained in a specialized database such as DOORS NG. VI's exported from DOORS NG were imported into Ingenium Dictionary Service to make them available for procedure authoring. FSW commands and telemetry are another type of VI's. For Europa and Psyche missions, they are managed in a Web application called Helix that was developed at JPL. FSW command and telemetry VI's were also exported from Helix and ingested into Ingenium. Two

types of steps are used to handle VI's in procedure. VI steps are used to list VI's that will be verified by the procedure. VI Status steps are used to define which steps are to verify which VI's. When a procedure is executed, verification statuses of VI's are derived from the pass/fail status of steps that are verifying VI's. These results provide a first order of data that supports V&V review.

Tagging Steps

It is common that a procedure is executed on venues of varying fidelity. For example, a dry run is done on a simulator only venue before the procedure is executed on a hardware venue for run for record (R4R). That means procedures need to account for differences of venues such as lacking capabilities in lower fidelity venues. A tagging capability was added to identify steps that are applicable only to specific test venues. When a procedure is imported, steps can be quickly filtered by tags. Another use case of tags is to mark different usage scenarios. For example, a testbed power on/off procedure may use "Power on" and "Power off" tags, and the procedure can be used to power on or power off test venue, or both as needed. Multiple tags can be applied to a single step. For example, "Testbed" tag and "Power on" tags can be applied to a step that will be used to power on a testbed venue.

Nested Procedures

Most functional test procedures depend on one or more utility procedure. One prime example is power on/off procedure. Ingenium provides a capability to refer to other procedures, in any level of depth when needed. A functional procedure may refer to the power-on portion of a power on/off procedure at the beginning, and refer to the power-off portion of a power on/off procedure at the end. A nested procedure can be executed in two ways. The first option is to import child procedure directly into the current procedure, where steps of the nested procedure will be embedded directly in the parent procedure. This approach will result in a single execution that contains the parent and child executions. The other option is to create a child execution for the nested procedure. This approach will result in separate as-runs for child procedures. Another use case of nested procedures is baseline tests, which is a collection of regression tests used by ATLO. For baseline tests, a coordinator procedure refers to dozens of functional test procedures. Tags are often used to quickly select a subset of tests to perform. For baseline tests, it is common to run nested procedures in separate executions so that as-runs can be reviewed by individual subsystem or domain experts.

Large Procedures

Size of Ingenium procedures varies from dozens of steps to thousands of steps. As more complicated and lengthy procedures were created, users experienced slow response particularly in Ingenium UI. Performance optimization was done to improve the responsiveness of UI. First, a moving window of steps was introduced to render only a subset of steps in the main canvas. As the user scrolls up or down, steps are added or removed dynamically in the viewing area. Second, more efficient state management scheme was adopted for some of the VueJS components to reduce the computational cost of UI. Third, the complete outline is still rendered to allow quick navigation when working on a large procedure. Procedure data is stored in Arango graph database [16]. Thanks to the efficient graph query, query speed for a given procedure is linear to the number of steps, not affected by the total number of steps in database. Overall, loading a large size procedure in UI takes a few seconds.

Custom Scripts

Ingenium provides about 20 step types out of the box that cover the most commonly used functionalities across various procedures and projects. However, there are cases where flight projects need to create custom steps. One good example is parsing data products to verify the behavior of flight system or science data, whose formats are specific to individual projects. Other examples include interacting with project specific S/W interfaces, or performing the same test pattern many times such as checking hundreds of power switches. For these cases, a custom script can be developed to execute a project developed script as a step in Ingenium. Since custom script step uses a file-based interface, any programming language can be used, although all custom scripts have been written in Python so far by Europa and Psyche projects. A number of security measures are in place to make sure only authorized scripts of specific versions can be executed through Ingenium.

Review Comments

Comments can be added to procedure to facilitate procedure reviews. Reviewers and authors can start a conversation on an open issue by adding comments to a step in a procedure. When a discussion thread reaches to a closure, it can be marked as resolved. To track open issues, conversations are displayed in a side panel along with their resolution status. Steps can be filtered so that only steps with comments are displayed in the main canvas.

Configuration Management

Procedure development is an iterative process that may require several revisions. For a given procedure, the latest state of the procedure is maintained in a working version. The current state of the working version can be saved as a new version at any time. These versions can be used as a waypoint or as a draft that will be sent out for a review. Only the working version can be edited, and versioned copies are immutable in Ingenium except for their meta data such as release status. Procedures are reviewed in Ingenium through review comments. Yet, procedure approval is conducted through a JPL institutional system called PBAT (Procedures for Build Assemble and Test). Since PBAT is a document-oriented procedure management and archival system, Ingenium procedures are exported in PDF format that looks similar to traditional paper procedures, and uploaded to PBAT for approval.

B. Procedure Execution

Multiple venues of various types can be registered in Ingenium. To execute a procedure, the user starts an execution on a venue, and import the procedure. It is possible to execute the same procedure on different venues.

Automatic Execution

By default, steps in Ingenium are executed one by one in manual mode. Users can turn on automatic mode to execute steps automatically. This allows running a big procedure in an efficient manner. It also helps when running a series of steps that depend on the precise timing of previous steps. Nested procedures are automatically imported in automatic mode. If a nested procedure is configured to run as a separate execution, execution will be automatically switched to the child execution, and will be switched back to the parent execution when the child procedure is completed. A number of helper facilities are available to control automatic execution. Automatic execution can be paused or aborted at any time. Breakpoints can be set at any steps or sections to pause at pre-defined locations. Rules can be set whether to stop automatic execution in the event of step failure, or step error, or at command steps.

Enforcement of Redline Approval

When a released procedure is executed for official test record (i.e., run for record, R4R mode), any changes to the procedure must be recorded and approved. Such modification can be tactical and temporary (blueline changes) in nature, for example to account for specific configuration of the test venue, or can be corrections to the procedure that would be incorporated to the next release of the procedure (redline changes). When a procedure is modified during a R4R execution, Ingenium automatically enforces the redline rules. Three types of redline change are supported: modification of a step, addition of a step, or deletion of a step. A redline step must be approved by the test conductor or by a participant who has the authority, before it can be executed.

In practice, many procedures are executed with redline or blueline modification due to changes in test configurations, changes in new FSW versions, or errors in procedures. Therefore, it is critical to track blueline/redline changes and make sure they are dispositioned properly, and redline changes are incorporated in the next release of the procedure. To support managing redline/blueline modification, a number of facilities were added to Ingenium. Redline/blueline steps are highlighted in red/blue colors in the outline side panel. When multiple steps are changed as redline or blueline, they can be updated and approved in a bulk operation. When a procedure is updated to incorporate redline changes, redline steps can be copied from as-run to replace the current steps in the procedure.

Watch Live Execution

Only a single user can execute steps for an execution at a given time. Other users can watch the execution in live by opening up the execution in a web browser. Results of step executions are available in real time to users who are watching execution. Watchers can also add comments to execution that will be visible to test conductor and other watchers. This bi-directional live update was implemented through web socket connections between Ingenium service and users' web browsers. Ingenium Core Service generates Web socket messages that are forwarded to Web browsers of test conductor and watchers. The format of the web socket messages is based on the RESTful API specification of Core service. This approach enables UI codes to process responses of the Core server in the same way through the RESTful API or through the Web Socket connection.

Review Comments

Comments can be added to steps by the test conductor or watchers while an execution is in progress. These comments are often used to capture anomalies that need to be investigated or dispositioned. After an execution is closed, review comments can be added to support data review or V&V review. When a comment thread reaches to a closure, it can be marked as resolved. To track open issues, status of comments (total count, closed count, and open count) is available for executions, and steps can be filtered by the existence of comments in the main canvas.

C. Review and V&V

Ingenium stores a large amount of data in the form of procedures and as-runs. While Ingenium provides ways to examine the data through web interface, individual projects may want to generate various specialized reports or integrate data in Ingenium with data in another source such as GDS database. A number of Python scripts were created that pull data from Ingenium and generate reports for procedure review and data review.

Procedure Validation Reports

This script checks a procedure version against a flight dictionary to flag steps that use commands and telemetries that do not match the definitions in the dictionary. This can be used to check if a procedure is still valid when a new version of flight dictionary is released. The script also checks VI's used in a procedure and flags any VI's that do not match the latest data in Dictionary Service.

Integrated Session Reports

Commands and telemetry data generated during tests are organized by session identifiers in GDS database. For each test activity, a session report is generated that combines all commands and telemetry data in time order to support V&V review. When paper based or scripted procedures are used, it is not easy to interpret a session report in the context of steps as defined in procedure. For Ingenium execution, the start and end time of step executions are readily available, which makes it possible to create a session report that includes Ingenium step executions. A Python script was developed to source both from Ingenium database and from GDS database, and generate integrated session reports. This allows examining command and telemetry data in the context of step execution timing and pass/fail status. While Ingenium procedure looks for telemetries that are specified in the procedure, an integrated session report provides a complete picture of the test, which is essential to accurately assess the behavior of the flight system and identify any anomalies during tests.

V&V Reports

Ingenium V&V report lists all VI's referenced in an as-run and summarizes their verification status based on steps' pass/fail statuses. If pass/fail status of a step has been overridden by test conductor, the overridden status will be used in the report. V&V reports give a high-level summary of verification status.

D. Operational Requirements

As Ingenium was used by testbed and ATLO teams on a daily basis, a number of enhancement requests came up that had significant impact on the day-to-day operation of the teams.

Suspend/Resume Execution

When an execution is started for a test venue, the venue is locked for the execution and Ingenium does not allow other executions active on the venue. If an execution is not completed during a shift and the venue needs to be handed over to the next shift, the execution can be suspended to release the venue for the next shift. This capability was critical for test bed operation, where hardware test venues are scarce resource that are shared by many V&V teams in a tightly managed shift schedule. The suspend/resume capability is also handy when a procedure takes more than one shift, where the execution can be suspended and resumed as many times as needed before it is completed. The same mechanism can be also used to hand over an execution to another user in the middle of a shift or to the next shift.

Operation Inside Mission Net

Due to security reasons, ATLO venues are set up inside the mission network that is not accessible from the regular JPL network. This means a dedicated instance of Ingenium needs to be installed for ATLO. Containerization of services allowed quickly setting up another Ingenium instance. Nonetheless, the fact that testbed and ATLO have separate instances of Ingenium posed a few challenges that required migration of procedures and as-runs between the instances.

Most of ATLO test procedures are developed and dry run on a testbed before it is executed on an ATLO venue. This required migration of released test procedures from a testbed Ingenium to ATLO Ingenium. An export/import capability was developed to migrate procedures across Ingenium instances. Similarly, a migration capability for as-run has been developed. When a procedure is executed on an ATLO venue, the as-run is not readily available for reviewers who have to log into the mission net. When Psyche ATLO moved from JPL in California to the launch site in Florida, ATLO Ingenium was moved as well, and the remote access to Ingenium from JPL was slow due to the limited network bandwidth. This caused challenges to V&V teams who need to review as-runs in the ATLO instance.

To make as-runs more accessible, a script was created that migrates as-runs across Ingenium instances. Using this capability, ATLO as-runs were copied over to the testbed Ingenium instance that is more easily accessible.

V. Application to Flight Projects

As of the time of this writing (Nov 2022), Psyche and Europa Clipper project teams are actively using Ingenium. Psyche testbed started using Ingenium in early 2019, and an ATLO instance was set up in 2020. Europa system testbed started using Ingenium in late 2019, and an ATLO instance was set up in early 2022. Between the two projects, more than fifteen hundred procedures were released in Ingenium, and close to twenty thousand executions have been performed so far.

A. Procedure Examples

Various types of procedures were created in Ingenium. Electronic integration procedures were used when testbeds were integrated and when flight system integration was started in ATLO. Utility procedures such as power on/off procedures were created early on to support testbed and ATLO operations. Numerous functional and non-functional V&V procedures were created. Psyche ATLO created a suite of regression tests using the tagging capabilities. To date, Psyche project released more than 700 procedures in Ingenium, and Europa Clipper project released more than 800 procedures. Both projects utilized a set of procedure templates to promote consistency and best practices in procedure developments.

Figure 3 displays four steps in an Ingenium procedure, illustrating a common test pattern for sequence activation and verification. The steps are as follows:

- 11-1-S-1 Check Sequence Activation Count**: This step checks the initial state of channel SEQ-0161 SEQ_ACTIVATION_SUCCESS (which keeps a count of the times a sequence has been successfully activated) and records the initial state. It includes a query from CURRENT_TIME (Lookback: 0 seconds, Timeout: 60 seconds) and a table showing SEQ-0161 SEQ_ACTIVATION_SUCCESS On TZ-A VALUE (DN) Record.
- 11-1-S-2 Activate Sequence**: This step sends commands to the spacecraft or simulator. It includes a query from LAST_STEP_END on TZ-A (Lookback: 30 seconds, Timeout: 120 seconds) and a table showing the command details: FSW CMD: [redacted] EPOCH_0-1/eng1/seq_time_type.seq, Verify: false, String: DEFAULT.
- 11-1-S-3 Wait for Sequence to Complete**: This step waits for the EVR's that confirm the sequence is done executing. It includes a query from LAST_STEP_END on TZ-A (Lookback: 30 seconds, Timeout: 120 seconds) and a table showing the command details: Name: SEQ_SVC_EVR_DONE_VALIDATION_TRANSITION_EXECUTING Messa..., EXISTS, Name: SEQ_SVC_EVR_DONE_EXECUTION Message: /eng1/seq_time_typ..., EXISTS.
- 11-1-S-4 Confirm Sequence Activation Count**: This step checks the state of channel SEQ-0161 after completing the sequence. If the sequence was activated successfully, this channel should have increased its count by 1 with respect to the recorded initial state. It includes a query from LAST_STEP_END (Lookback: 0 seconds, Timeout: 60 seconds) and a table showing SEQ-0161 SEQ_ACTIVATION_SUCCESS On TZ-A CHANGE (DN) = 1.

Figure 3: Command and telemetry steps in Ingenium and a common test pattern (Note: command name was redacted).

What sets apart e-procedures from paper procedures is the ability to send commands and check telemetry directly within procedure. Figure 3 is an example of command and telemetry steps from a sequence V&V procedure in Ingenium, where the behavior of sequence activation is verified. A common test pattern of flight system is also described in the figure. First, the initial state of the system is recorded by querying channelized telemetry using Verify EHA Step. While this step can be omitted in certain cases, it is required to verify the change of channel values. Second,

send one or more FSW or SSE command. If “Verify” option is on, the step will wait until the successful completion of the commands is confirmed. Third, wait for EVRs specific to the command, which is another evidence that the command was properly executed. Fourth, verify that state variables reached expected values using Verify EHA or Wait EHA step. This will confirm that the command achieved the intended effects. This pattern is repeated in many procedures.

B. Procedure Executions

Figure 4 shows the accumulated number of procedure executions in Psyche testbed and ATLO. Over the last three years, Psyche testbed performed close to ten thousand executions, including runs on WSTS. About 10% of the executions (~1000 as of Nov 2022) were run for record. This clearly shows that procedure development is an iterative process that requires trial and error, and many developmental and dry runs. In ATLO, more than sixteen hundred executions were completed, and about 90% of them (1476 as of Nov 2022) were run for record. During the first half of 2022, Psyche testbed was running about 20 executions per day and ATLO was running about 6 executions per day. Since the launch of the Psyche mission was pushed to October 2023, Psyche ATLO paused its operation for a re-plan, but V&V activities continue in testbed. The ATLO operation is expected to resume in late 2022.

Figure 5 shows a similar execution data of Europa testbed and ATLO. So far, Europa testbed performed more than seven thousand executions, including runs on WSTS. About 20% of the executions (~1600 as of Nov 2022) were run for record. In ATLO, more than three hundred executions were completed, and about 90% of them (289 as of Nov 2022) were run for score. During the last six months, Europa testbed was running about 25 executions per day and ATLO was running about 2 executions per day. Europa testbed is expected to continue in this pace for a while, and ATLO activities are expected to pick up over the next few months. Overall, Ingenium has been successfully supporting both Psyche and Europa testbed and ATLO operations throughout their tight shift schedules (double shifts and triple shifts in some occasions).

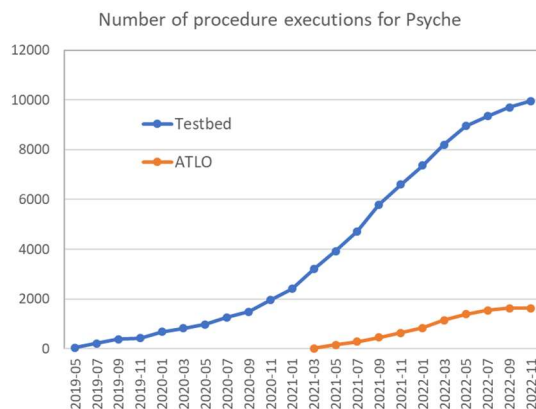


Figure 4: Execution counts for Psyche.

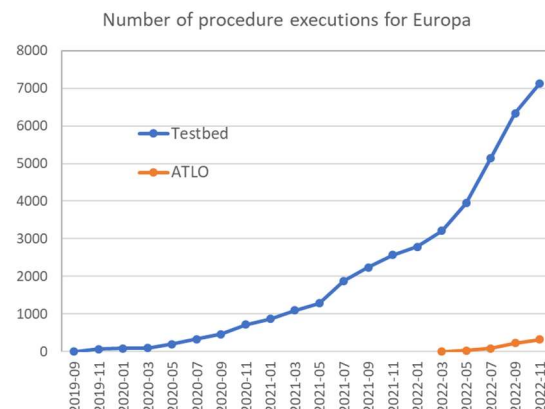


Figure 5: Execution counts for Europa Clipper.

C. Impacts to Testbed and ATLO Operations

Ingenium brought in significant improvements to the process around test procedure authoring, execution, and reviews of JPL flight projects. First, it provided a unified way to craft test procedures. Since test procedures are composed of common step types as building blocks, procedures can be more easily understood and impacts of steps are more predictable. Second, since Ingenium provides a unified and high-level interface to GDS and SSE systems, it lowered the entry barriers to I&T and V&V for new engineers, who have to gain knowledge in multiple subjects such as GDS, SSE, and flight systems to be effective. Third, mistakes were reduced during procedure execution since users do not need to copy and paste commands from a document or manually check telemetry by browsing through live telemetry in GDS windows. Since the same procedure can be dry-run in a lower fidelity venue, more issues in a procedure can be addressed before it gets to a higher fidelity venue. Fourth, Ingenium streamlined the review process of procedures and as-runs since they are readily available online, along with discussion threads of open issues. Fifth, Ingenium procedures can be more easily reused across teams in a project and also across projects, since procedures are in the same format.

Most of the impacts mentioned above are applicable to ATLO operations. According to Psyche ATLO team members, they were executing procedures in ATLO in a much more efficient way than they did in the past. Typically,

three key personnel are present when executing an ATLO procedure: test conductor, command, and system. A test conductor orchestrates the overall activities. A command personnel is responsible for sending FSW and SSE commands. A system personnel is responsible for checking the behavior of flight system during tests. Since Ingenium automates much of the commanding and telemetry checking, they could now run with two people (test conductor and command). Yet, Psyche ATLO continues to operate with three personnel, as it adds operational flexibility and Ingenium lets them pay more attention to the test content instead of time-consuming and error-prone manual checks.

D. Future Applications

Beyond applications to testbed and ATLO, Europa Clipper project is planning to use Ingenium for its operational procedures. This means procedures of the same format can be used to test flight systems and also to operate flight systems, and operational procedures may reuse contents from test procedures. This approach is another way to improve the reliability of flight missions through the TAYF (test as you fly) principle.

Sample Return Lander (SRL) is the next NASA/JPL flagship mission that is planning to use Ingenium for its testbed and ATLO operations. SRL is also interested in using Ingenium for sub-system level tests, where the system under test is exercised by a specialized command and telemetry system rather than AMPCS. Ingenium team is investigating ways to extend Ingenium to support non-AMPCS test venues, possibly through the use of custom scripts.

VI. Conclusions

Ingenium is a solution that supports the entire lifecycle of test procedures of JPL flight projects. It combines the advantages of paper procedures (flexibility and human friendliness) and scripted procedures (rigor and speed). It streamlines the process around procedure creation, execution, and review. It removes time consuming manual checks so that test engineers can reduce errors and execute test procedures more efficiently. Its unified approach lowers entry barriers to I&T and V&V for new engineers, and allows reuse of procedures across teams and projects. Ingenium integrates command and telemetry dictionary contents and V&V contents, which makes it easy to develop V&V procedures. Automatic enforcement of redline/blueline modifications were crucial to be able to execute test procedures in a rigorous way that satisfies JPL policy.

Ingenium has been used successfully by Psyche and Europa Clipper testbed and ATLO teams. So far, more than fifteen hundred procedures were released in Ingenium, and more than forty-five hundred run-for-record executions have been completed. The same procedure can be executed in testbed before ATLO, which substantially lowered the chance of finding issues during ATLO tests. ATLO team was able to execute procedures in a much more efficient way than they did in the past. Since Ingenium automated much of the commanding and telemetry checking, ATLO personnel could pay more attention to test contents rather than time consuming manual checks.

Development of Ingenium is on-going, keeping up with new requirements and use cases as it is heavily used by JPL flight projects. Application of Ingenium to subsystem level test is being investigated, where non-standard interface to test venue needs to be integrated. The procedure and as-run repository of Ingenium already has wealth of data, which can be queried through its API for data mining. Since all historical data is stored in a searchable format, answers to challenging questions become possible such as “which test procedures use this particular command?”, “what are the cases where this command failed?”, or “how long did it take to complete power-on procedure on average?”. Some of these metrics may provide insights that can be directed to improve I&T process and also Ingenium capabilities.

Acknowledgments

The work described in this paper was carried out at the Jet Propulsion Laboratory (JPL), California Institute of Technology, using JPL internal funds through Integrated Model Centric Engineering (IMCE). The Jet Propulsion Laboratory is under a contract with the National Aeronautics and Space Administration, and U.S. Government sponsorship is acknowledged.

References

- [1] Levesque, M., Louie, J., and Guerrero, A., “Test Execution Control Tool: Automating Testing in Spacecraft Integration and Test Environments,” IEEE Aerospace Conference, 2000.
- [2] Gibbs, D., and Bone, B., “Testing Flight Systems with Machine Executable Scripts,” IEEE Aerospace Conference, March 7-14, 2009.
- [3] Choi, J., “Cost-Effective Telemetry and Command Ground Systems Automation Strategy for the Soil Moisture Active Passive (SMAP) Mission,” AIAA SpaceOps Conference, 2012.
- [4] <https://robotframework.org>, cited in May 2022

- [5] Nagendra, M., Chinnaswamy, C. N., and Sreenivas, T. H., "Robot Framework: A boon for Automation," *International Journal of Scientific Development and Research*, Vol. 3, No. 11, 2018, pp. 446-449.
- [6] <https://jupyter.org>, cited in May 2022.
- [7] Lasser, J., "Creating an Executable Paper is a Journey through Open Science," *Communications Physics*, Vol. 3, 2020, article 143.
- [8] Bell, S., and David Kortenkamp, "Embedding Procedure Assistance into Mission Control Tools," Proceedings of the International Joint Conference on Artificial Intelligence Workshop on AI in Space, 2011.
- [9] Mary Beth Hudson, Arthur Molin, and David Kortenkamp. Electronic procedures for medical operations in space. In Proceedings of the AIAA Space 2011 Conference and Exposition, 2011.
- [10] <https://www.epsilon3.io>, cited in Nov 2022.
- [11] <https://vuejs.org>, cited in May 2022.
- [12] <https://www.openapis.org>, cited in May 2022
- [13] <https://docs.docker.com>, cited in May 2022
- [14] <https://docs.docker.com/engine/swarm/>, cited in May 2022
- [15] Dehghani, N., Tankenson, M., "A Multi-mission Event-Driven Component-Based System for Support of Flight Software Development, ATLO, and Operations first used by the Mars Science Laboratory (MSL) Project," SpaceOps 2006 Conference, Rome, Italy, Jun 19-23, 2006.
- [16] <https://www.arangodb.com/community-server/native-multi-model-database-advantages>, cited in Nov 2022